Materia: Programación IV

UNIDAD 7 · SERVER STATE Y NAVEGACIÓN

APUNTE TEÓRICO

Mutaciones con React Query

Revisor de la unidad: Prof. Franco González

Introducción

En muchas aplicaciones web es necesario **modificar datos en el servidor**.
Por ejemplo:

- Crear un nuevo elemento
- Editar un elemento existente
- Eliminar un elemento

En **TanStack Query** (antes React Query) estas operaciones se realizan mediante **mutaciones**, con el hook **useMutation**.

A diferencia de **useQuery**, que se usa para **obtener datos**, **useMutation** se usa para **modificar datos**. Además, `useQuery` se ejecuta automáticamente cuando el componente se monta, mientras que `useMutation` no se ejecuta por sí solo: la operación se dispara de forma explícita, por ejemplo al enviar un formulario, mediante la función `mutate`.

¿Qué es una mutación?

Una **mutación** es una operación que cambia datos en el servidor.

Ejemplos comunes:

- Crear un usuario
- Editar una tarea
- Eliminar un producto
- Actualizar un perfil

Estas operaciones suelen realizarse mediante solicitudes HTTP con los métodos:

- `POST`
- `PUT`
- `PATCH`
- `DELETE`

Hook useMutation

El hook `useMutation` permite ejecutar una operación que modifica datos en el backend.

Ejemplo básico:

```
import { useMutation } from "@tanstack/react-query";

type Tarea = { id: number; title: string };
type NuevaTarea = { title: string };

const mutation = useMutation({
  mutationFn: async (nuevaTarea: NuevaTarea): Promise<Tarea> => {
    const response = await fetch("/api/tasks", {
      method: "POST",
      body: JSON.stringify(nuevaTarea),
      headers: {
        "Content-Type": "application/json",
      },
    });
    if (!response.ok) {
      throw new Error("No se pudo crear la tarea");
    }
    return response.json();
  },
});
```

Como `fetch` no lanza un error cuando el servidor responde con un código HTTP de error (4xx o 5xx), la función comprueba `response.ok` y, si la respuesta no es correcta, lanza un error; así la mutación pasa al estado de error y se ejecuta `onError`. Si la respuesta es correcta, la función devuelve los datos recibidos mediante `response.json()`. Los tipos `NuevaTarea` (los datos que se envían) y `Tarea` (la tarea que devuelve el servidor) le indican a TypeScript qué recibe `mutate` y qué devuelve la mutación; si el parámetro de `mutationFn` no se tipa, TypeScript no admite argumentos en `mutate`.

Para ejecutar la mutación se utiliza:

```
mutation.mutate({ title: "Repasar useMutation" });
```

Estados de una mutación

Las mutaciones tienen distintos estados que permiten adaptar la interfaz de usuario a cada etapa de la operación.

Los principales son:

- **isPending** → la operación está en progreso
- **isSuccess** → la operación se completa correctamente y la respuesta está disponible en `mutation.data`
- **isError** → la operación produce un error, disponible en `mutation.error`

Estos estados permiten informar al usuario sobre el avance y el resultado de la operación: mientras `isPending` es verdadero, se suele mostrar un indicador de carga y deshabilitar el botón de envío (por ejemplo, con `disabled={mutation.isPending}`) para evitar envíos duplicados; con `isSuccess` se muestra una confirmación y con `isError`, un mensaje de error. Antes de ejecutarse, la mutación está en estado `"idle"` (`isIdle` es verdadero). Los cuatro estados también se consultan mediante `mutation.status`, que toma los valores `"idle"`, `"pending"`, `"success"` o `"error"`.

onSuccess y onError

React Query permite ejecutar lógica después de que una mutación termina.

onSuccess

Se ejecuta cuando la operación se completa correctamente. Recibe como primer argumento los datos que devuelve `mutationFn` (por ejemplo, la tarea creada por el servidor) y como segundo, las variables enviadas a `mutate`. En los ejemplos siguientes, `crearTarea` es la función que realiza la petición `POST` al backend, con la misma estructura y los mismos tipos que el `mutationFn` del ejemplo básico: recibe una `NuevaTarea` y devuelve una `Promise<Tarea>` con la tarea creada.

Ejemplo:

```
const mutation = useMutation({
  mutationFn: crearTarea,
  onSuccess: (tareaCreada) => {
    alert(`Tarea "${tareaCreada.title}" creada correctamente`);
  },
});
```

Usos comunes:

- Mostrar un mensaje de éxito
- Limpiar formularios
- Actualizar datos en pantalla

onError

Se ejecuta cuando ocurre un error en la mutación.

Ejemplo:

```
const mutation = useMutation({
  mutationFn: crearTarea,
  onError: (error) => {
    alert(`Ocurrió un error al crear la tarea: ${error.message}`);
  },
});
```

Esto permite manejar errores de forma controlada.

Invalidación de Queries

Cuando se modifican datos en el servidor, los datos almacenados en la caché pueden quedar **desactualizados**.

Por ejemplo:

1. La interfaz muestra una lista de tareas.
2. Se crea una nueva tarea.
3. La lista debe reflejar la tarea creada, pero sigue mostrando los datos almacenados en la caché.

Para esto se utiliza **invalidateQueries**.

Ejemplo de invalidación

```
import { useMutation, useQueryClient } from "@tanstack/react-query";

const queryClient = useQueryClient();

const mutation = useMutation({
  mutationFn: crearTarea,
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["tasks"] });
  },
});
```

Esto marca como desactualizadas (stale) las consultas cuya clave comienza con `["tasks"]`. La lista de tareas se obtiene con `useQuery({ queryKey: ["tasks"], queryFn: obtenerTareas })`, donde `obtenerTareas` es la función que realiza la petición `GET` al backend y devuelve una `Promise<Tarea[]>`. Como esa consulta comparte la clave invalidada y está en pantalla, TanStack

Query la vuelve a solicitar en segundo plano y la lista se actualiza con los datos del servidor. Si se omite la invalidación, la tarea se guarda en el servidor, pero la lista sigue mostrando los datos almacenados en la caché.

Ejemplo completo: Crear tarea

```
const queryClient = useQueryClient();

const crearTareaMutation = useMutation({
  mutationFn: async (tarea: NuevaTarea): Promise<Tarea> => {
    const response = await fetch("/api/tasks", {
      method: "POST",
      body: JSON.stringify(tarea),
      headers: {
        "Content-Type": "application/json",
      },
    });
    if (!response.ok) {
      throw new Error("No se pudo crear la tarea");
    }
    return response.json();
  },

  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["tasks"] });
  },

  onError: () => {
    alert("Error al crear la tarea");
  },
});
```

Uso en un formulario:

El componente requiere importar `useState` y el tipo `SyntheticEvent` desde `react`, y `useQuery` desde `@tanstack/react-query`. Las declaraciones de `queryClient` y `crearTareaMutation` del ejemplo anterior se ubican al comienzo del componente, porque los hooks solo pueden llamarse dentro de un componente. La lista usa la misma clave que la invalidación y el botón se deshabilita mientras la mutación está en curso.

```
function CrearTarea() {
  const [title, setTitle] = useState("");
  // queryClient y crearTareaMutation, como en el ejemplo anterior
  const { data: tareas } = useQuery({
    queryKey: ["tasks"],
    queryFn: obtenerTareas,
  });

  const handleSubmit = (e: SyntheticEvent<HTMLFormElement>) => {
    e.preventDefault();
    if (!title.trim()) return;
    crearTareaMutation.mutate({ title });
  };

  return (
    <>
      <form onSubmit={handleSubmit}>
        <input value={title} onChange={(e) => setTitle(e.target.value)} />
        <button type="submit" disabled={crearTareaMutation.isPending}>
          {crearTareaMutation.isPending ? "Creando..." : "Crear tarea"}
        </button>
      </form>
      <ul>
        {tareas?.map((tarea) => (
          <li key={tarea.id}>{tarea.title}</li>
        ))}
      </ul>
    </>
  );
}
```

Editar y eliminar siguen el mismo patrón: cambian el método HTTP (`PUT` o `PATCH` para editar, `DELETE` para eliminar) y la URL, que incluye el identificador del recurso (por ejemplo, `/api/tasks/5`), y en `onSuccess` se invalida la misma clave `["tasks"]` para que la lista refleje el cambio.

Conclusión

`useMutation` ejecuta los cambios en el servidor cuando la aplicación lo indica mediante `mutate`, sus estados permiten informar al usuario el avance y el resultado de la operación, y la invalidación de la clave correspondiente mantiene sincronizados con el servidor los datos que muestra `useQuery`.

Bibliografía

1. React. (s. f.). *React documentation*. https://react.dev
2. TanStack. (s. f.). *TanStack documentation*. https://tanstack.com